CS221 Fall 2025: Artificial Intelligence: Principles and Techniques

Homework 2: Sentiment

SUNet ID: [your SUNet ID]
Name: [your first and last name]
Collaborators: [list all the people you worked with]

By turning in this assignment, I agree by the Stanford honor code and declare that all of this is my own work.

Problem 1: Building Intuition for Bag-of-Words Features and Linear Classification

Social media platforms like Twitter are rich sources of emotional expression. In this homework, you'll build classifiers to detect emotions in tweets using linear classification with cross-entropy loss and softmax activation. Consider the following dataset of 6 tweets, each labeled with exactly one emotion: **joy** (J), **anger** (A), or **fear** (F):

Tweet	Emotion	One-hot encoding
"amazing day"	Joy	[1, 0, 0]
"scared of spiders"	Fear	[0, 0, 1]
"love this"	Joy	[1, 0, 0]
"so angry"	Anger	[0, 1, 0]
"so so worried about tomorrow"	Fear	[0, 0, 1]
"hate waiting"	Anger	[0, 1, 0]

Each tweet x is mapped to a feature vector $f(x)$ using **bag-of-words representation** (you can think of it as word counts). Machine learning models don't directly take a string as input; instead, as you have learned, they work with matrices (like NumPy arrays and tensors). One way to convert strings to tensors is "bag-of-words" features, which treat input texts as a "bag of words" and represents them using a vector with the same length as our vocabulary. The resulting representation is a vector, where each position corresponds to a word in the vocabulary, and each value represents how many times that word appears in the input text. If we have a very large vocabulary, we would end up with a very sparse vector with many 0's.

For this problem, let's assume our vocabulary consists of all unique words in the tweets above: **{about, amazing, angry, day, hate, love, of, scared, so, spiders, this, to-**

morrow, waiting, worried}.

To build our multi-class classifier, we use a weight matrix $\mathbf{W} \in \mathbb{R}^{d \times 3}$ where d is the feature dimension (vocabulary size) and 3 is the number of classes. The model computes logits as $\mathbf{z} = \mathbf{W}^T f(x)$ and applies softmax to convert logits into probability values that sum to 1. Let $\mathbf{p}$ be the 3-dimensional vector of output probabilities, and p_k be the probability that the kth element of p has a true label of 1. The softmax probabilities are computed in the following way:

$$p_k = \frac{e^{z}k}{\sum_{j=1}^{3} e^{z}j}$$

Then, given the softmax probabilities, the classifier will use argmax to choose the prediction class with the highest p_k.

The cross-entropy loss measures the difference between a model's predicted probabilities and the true probability distribution of the data. For a single example (with three output classes), it can be computed as follows:

$$\text{Loss}_{\text{CE}}(x, \mathbf{y}, \mathbf{W}) = -\sum_{k=1}^{3} y_k \log p_k$$

where $\mathbf{y}$ is the one-hot encoded true label vector.

Part 1a: Feature Representation

Question: Using the vocabulary given above, write out the bag-of-words feature vector $f(x)$ for the tweet "so so worried about tomorrow".

What we expect: A feature vector corresponding to the vocabulary in alphabetical order: [about, amazing, angry, day, hate, love, of, scared, so, spiders, this, tomorrow, waiting, worried].

Your Solution: $[1, 0, 0, 0, 0, 0, 0, 0, 2, 0, 0, 1, 0, 1].T$

Part 1b: Softmax Computation

Question: Given logits $\mathbf{z} = [2.0, 1.0, -1.0]$ for the three classes [Joy, Anger, Fear], compute the softmax probabilities. Show your work.

What we expect: The softmax probability vector $[P(\text{Joy}), P(\text{Anger}), P(\text{Fear})]$ with values rounded to 3 decimal places.

Your Solution: $[0.705, 0.259, 0.035].T$

Part 1c: Cross-entropy Loss

Question: For the tweet "so angry" with true label Anger (one-hot: [0, 1, 0]), suppose your model outputs probabilities $[0.2, 0.7, 0.1]$. Calculate the cross-entropy loss for this example. Then explain what happens to the loss as the predicted probability for the correct class approaches 1.

What we expect:

1. The numerical cross-entropy loss value
2. A brief explanation (2-3 sentences) of the loss behavior

Your Solution: $Loss = 0.357$
When the probability of the correct class approaches 1, the total loss approaches 0.

Part 1d: Gradient Analysis

Question: Derive $\frac{\partial \text{Loss}_{\text{CE}}}{\partial z_k}$, the gradient of the cross-entropy loss with respect to the logit z_k. Show the mathematical steps to find $\frac{\partial \text{Loss}_{\text{CE}}}{\partial z_k}$ and express your final answer in terms of p_k and y_k only. Then, explain intuitively why this expression makes sense for gradient-based learning.

What we expect:

1. Mathematical derivation showing the steps to compute $\frac{\partial \text{Loss}_{\text{CE}}}{\partial z_k}$
2. Final gradient expression in terms of p_k and y_k
3. A brief intuitive explanation of why this gradient expression makes sense for learning (2-3 sentences)

Your Solution:

$$\begin{aligned}
\frac{\partial Loss_{CE}}{\partial z_k} &= \frac{\partial}{\partial z_k}\left(-\sum_{m=1}^{3} y_m \log p_m\right) \\
&= -\sum_{m=1}^{3} y_m \frac{\partial \log p_m}{\partial z_k} \\
&= -\sum_{m=1}^{3} \frac{y_m}{p_m} \frac{\partial \frac{e^{z_k}}{\sum_{m=1}^{3} e^{z_m}}}{\partial z_k} \\
&= \sum_{m=1}^{3} y_k(p_k - 1)
\end{aligned}$$

Problem 2: Building Intuition for Embeddings and Multilayer Perceptron

In the previous problem, we used bag-of-words features for sentiment classification. It is now an outdated method of representing texts in machine learning due to many limitations. For example, it doesn't capture word relationships (e.g., "amazing" and "wonderful" are similar but treated as completely different features) and creates very sparse, high-dimensional vectors. In this problem, we'll explore how neural networks with word embeddings can address these issues.

We'll use the same set of tweets from Problem 1, but now represent each word with a dense 2-dimensional embedding vector. For simplicity, assume we have the following pre-trained word embeddings:

Word	Embedding
amazing	[0.8, 0.6]
day	[0.2, 0.1]
scared	[-0.5, -0.7]
of	[0.0, 0.0]
spiders	[-0.3, -0.4]
love	[0.9, 0.4]
this	[0.0, 0.0]
so	[0.1, 0.0]
angry	[-0.6, -0.8]
worried	[-0.3, -0.6]
about	[0.0, -0.1]
tomorrow	[0.2, -0.2]
hate	[-0.8, -0.5]
waiting	[-0.1, -0.3]

For simplicity, we will focus on **binary sentiment classification (positive or negative)** instead of three-class classification for this problem only.

Part 2a: Tweet Representation via Averaging

Question: One simple approach is to represent each tweet as the average of its word embeddings. Compute the averaged embedding representation for "so angry". Comment on one advantage and one disadvantage of this approach.

What we expect:

1. A 2-dimensional averaged embedding vectors
2. Discuss one benefit and one limitation of creating embeddings for a text by averaging embeddings of its component words (2-3 sentences)

Your Solution: $[0.2, -0.4].T$
Benifit: easy to compute;
Limitation: easy to be influced by irrelavant words.

Part 2b: Forward Pass

Question: Using the averaged representation for "so angry" from part (a), perform a forward pass step-by-step to find the predicted label $\hat{y}$. Show your intermediate calculations by filling in the table below.

For the sake of simplicity, we'll use a 2-input, 2-hidden neuron, 1-output architecture consisting of:

1. **Input layer:** Word embeddings $\mathbf{x}$
2. **Hidden layer:** 2 neurons with ReLU activation, where $\text{ReLU}(\mathbf{x}) = \max(0, \mathbf{x})$
3. **Output layer:** 1 neuron (binary sentiment classification) with sigmoid activation

The forward pass equations are:

$$\mathbf{h} = \text{ReLU}(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)})$$

$$z = \mathbf{W}^{(2)}\mathbf{h} + b^{(2)}$$

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

where $\mathbf{x} = [x_1, x_2]^T$ is the 2D input representation, here $\mathbf{x} = [x_1, x_2]^T$ is the 2D input representation, $\hat{y}$ is the predicted label, $\mathbf{W}^{(1)} \in \mathbb{R}^{2\times 2}$, $\mathbf{b}^{(1)} \in \mathbb{R}^2$, $\mathbf{W}^{(2)} \in \mathbb{R}^{1\times 2}$, and $b^{(2)} \in \mathbb{R}$.
Network parameters:

$$\mathbf{W}^{(1)} = \begin{bmatrix} 1.0 & 0.5 \\ -0.5 & 1.0 \end{bmatrix}, \quad \mathbf{b}^{(1)} = \begin{bmatrix} 0.1 \\ -0.2 \end{bmatrix}$$

$$\mathbf{W}^{(2)} = \begin{bmatrix} 0.8 & -0.6 \end{bmatrix}, \quad b^{(2)} = 0.3$$

You should write your answers by copying and pasting the given table and filling its cells, or using bullet points to clearly state the value of each cell.

What we expect: Compute forward pass values at each node. Show your calculations for each step by filling in the table above.

Node/Variable	Formula/Computation	Value
$\mathbf{x}$	NA	$\begin{bmatrix}-0.25\\-0.4\end{bmatrix}$
$\mathbf{h}$	$\mathbf{h} = \text{ReLU}(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)})$	$\begin{bmatrix}0.0\\0.0\end{bmatrix}$
z	$z = \mathbf{W}^{(2)}\mathbf{h} + b^{(2)}$	0.3
$\hat{y}$	$\hat{y} = \sigma(z) = \frac{1}{1+e^{-z}}$	0.574

Your Solution:

Part 2c: Backward Pass

Question: Using the same weights, bias and input vector as in part (b), first compute the loss value L. Then, perform backpropagation to compute gradients $\frac{\partial L}{\partial \mathbf{W}^{(1)}}$, $\frac{\partial L}{\partial \mathbf{b}^{(1)}}$, $\frac{\partial L}{\partial \mathbf{W}^{(2)}}$, and $\frac{\partial L}{\partial b^{(2)}}$. Assume the true label is $y_{\text{true}} = 0$ (negative sentiment) and we're using binary cross-entropy loss:

$$L = -[y_{\text{true}} \log(\hat{y}) + (1 - y_{\text{true}}) \log(1 - \hat{y})]$$

You should write your answers by copying and pasting the tables below and filling their cells, or using bullet points to clearly state the value of each cell.

Variable	Formula/Computation	Value
L	$L = -[y_{\text{true}} \log(\hat{y}) + (1 - y_{\text{true}}) \log(1 - \hat{y})]$	0.853

Gradient	Formula (Chain Rule)	Evaluated Value
$\frac{\partial L}{\partial \hat{y}}$	$\frac{y_{true} - \hat{y}}{\hat{y}(\hat{y}-1)}$	2.347
$\frac{\partial L}{\partial z}$	$\frac{\partial L}{\partial y}\frac{\partial y}{\partial z}$	0.574
$\frac{\partial L}{\partial \mathbf{h}}$	$\frac{\partial L}{\partial y}\frac{\partial y}{\partial z}\frac{\partial z}{\partial \mathbf{h}}$	$\begin{bmatrix}0.459\\-0.344\end{bmatrix}$
$\frac{\partial L}{\partial \mathbf{W}^{(2)}}$	$\frac{\partial L}{\partial y}\frac{\partial y}{\partial z}\frac{\partial z}{\partial \mathbf{W}^{(2)}}$	$\begin{bmatrix}0.000\\0.000\end{bmatrix}$
$\frac{\partial L}{\partial b^{(2)}}$	$\frac{\partial L}{\partial y}\frac{\partial y}{\partial z}\frac{\partial z}{\partial \mathbf{b}^{(2)}}$	0.574
$\frac{\partial L}{\partial \mathbf{W}^{(1)}}$	$\frac{\partial L}{\partial y}\frac{\partial y}{\partial z}\frac{\partial z}{\partial \mathbf{h}}\frac{\partial \mathbf{h}}{\partial \mathbf{W}^{(1)}}$	$\begin{bmatrix}0.000 & 0.000\\0.000 & 0.000\end{bmatrix}$
$\frac{\partial L}{\partial \mathbf{b}^{(1)}}$	$\frac{\partial L}{\partial y}\frac{\partial y}{\partial z}\frac{\partial z}{\partial \mathbf{h}}\frac{\partial \mathbf{h}}{\partial \mathbf{b}^{(1)}}$	$\begin{bmatrix}0.000\\0.000\end{bmatrix}$

What we expect:

1. Compute the loss value L
2. Starting from $\frac{\partial L}{\partial \hat{y}}$, compute gradients for each node working backwards and fill out the tables given above. For the formula column, you should express each partial in terms of partials above it in the table (recall backpropagation from lecture!)
3. Feel free to reuse values you already computed in part b

Your Solution:

Part 2d: Embedding Analysis

Question: Looking at the provided word embeddings, identify patterns in how positive emotion words, negative emotion words, and neutral words are positioned in the 2D embedding space. You don't have to plot anything for this problem; simply observe the patterns. What does this suggest about how word embeddings capture semantic relationships?

What we expect:

1. Describe the spatial clustering of different word types (1-2 sentences)
2. Explain how embeddings capture semantic similarity and its advantage over bag-of-words (1-2 sentences)

Your Solution:
1. Positive emotion words (e.g., amazing, love) cluster in the first quadrant (positive coordinates), negative emotion words (e.g., scared, hate, angry) cluster in the third quadrant (negative coordinates), and neutral function words (e.g., this, about, of) lie near the origin (coordinates close to zero), forming distinct semantic clusters in the 2D embedding space.
2. Word embeddings encode semantic similarity via distance and direction in vector space: semantically similar words (e.g., amazing and love) are positioned close together, while opposites (e.g., love and hate) point in opposite directions. In contrast, the bag-of-words model only counts word frequencies and ignores semantic relationships, making embeddings far more meaningful for capturing the nuanced meaning of language.

Problem 3: Linear Classification

Part 3h: Exploring Learning Rates

Question: If you adjust `lr`, the learning rate, how do you expect loss and validation accuracy to change during training? Run the terminal command
`python submission.py --model linear --lr [your_learning_rate]`
to experiment with three different learning rates. Report what they are and what training behaviors you observed. Why did the learning rate affect loss and accuracy this way?

For example, you can run: `python submission.py --model linear --lr 0.1`. If you don't include an `--lr` argument, the default learning rate is 0.2.

What we expect: In 2-3 sentences, report three different learning rates you experimented with, and describe how they affected training behaviors. Explain why the learning rate affects loss and accuracy this way.

Your Solution: